Transforming semantic embedding functions for agentic traces

Helivan Jataware Johns Hopkins

Summer 2026

The problem

Modern generative systems produce logs of their actions and decisions— referred to as an *agentic trace*—while processing information. Traces often implicitly or explicitly encode information such as the particular type of tool that was used, the harness, etc. that may or may not be useful for determining if the trace produced a useful output. Further, the signal from using an off-the-shelf embedding function to encode trace similarity is often dominated by information that is potentially irrelevant for correctness, such as the harness or model used: in our corpus, nearest neighbors of raw trace embeddings are same-system 68–83% of the time, same-scaffold at 5× chance, and the exact agent configuration is retrievable at 0.82 vs. 0.43 chance even among same-model, same-harness siblings—while agreement with outcome sits at chance (AUC 0.52).

The goal of this work is simple: **develop a method for embedding agentic traces that better captures similarity of the work performed.** In developing such a method, we have identified six desirable properties of an agentic trace embedding, organized into three groups:

Faithful to content — what happened should drive similarity.

- **Task fidelity**: traces of different systems working the same problem should be similar. *Measured by* cross-system same-task nearest-neighbor retrieval.
- **Behavioral fidelity**: among traces of the same problem, similar approaches and results should be similar. *Measured by* within-task coherence of same-outcome traces (AUC of same- vs. different-outcome distances).

Invariant to authorship — who produced it should not.

- **Harness invariance**: scaffold formatting and tool idiom should not drive similarity. *Measured by* within-task same-scaffold retrieval vs. chance.
- **Model-family invariance**: the vendor/lineage of the underlying LLM should not drive similarity. *Measured by* within-task same-family retrieval vs. chance.
- **Identity invariance**: the exact system configuration (version, prompts, settings) should not be recoverable. *Measured by* exact-submission retrieval among same-model, same-harness siblings vs. chance.

Reliable — similarity should be a measurement, not noise.

- **Stability**: the embedding should be stable under agent stochasticity, judge resampling, and embedder choice. *Measured by* replicate-run and judge-resample agreement, per trace and aggregated over instances.

qubric

Our method for embedding agentic traces is based on a query-specific rubric. In particular, for each task instance an LLM (Model 1) reads the public problem statement and writes a query-specific rubric comprised of 1) understanding, 2) localization, 3) reproduction, 4) editing, 5) verification, and 6) final state. Given a trace corresponding to the task, a judge LLM (Model 2) extracts a short factual description per section. The short descriptions are then embedded with an off-the-shelf embedding model, centered against the cross-system median per instance, and concatenated. This approach enforces the equivalence relations the space should respect by respecting the rubric and using a fixed judge LLM as the extractor.

Evidence

We evaluate qubric on 107 system configurations from the SWE-bench Verified leaderboard (a 500-task benchmark; qubric judging is on a 20-task panel, with the 150-task scale-up in progress). We use gpt-5.4-mini for Model 1 and gpt-5.4-mini for Model 2 in the primary configuration; robustness to Model 2 is reported below. We compare qubric along the six axes described above against four baselines:

- **raw trace**: embed the first/last 8K tokens of the rendered trace directly—the off-the-shelf default;
- **free-form summary**: Model 2 summarizes the trace with no rubric—tests whether structure matters;
- **generic rubric**: the same six sections, fixed across instances—tests whether query-specificity matters;
- **verdict questions**: the rubric phrased as evaluative questions with short answers—tests descriptions vs. verdicts;

across **seven off-the-shelf embedding functions** spanning three families and two orders of magnitude of size. Per-pillar results for the baselines are shown in Fig. 2.

Qubric produces the same profile shift under all seven embedders (Fig. 1): *task fidelity* is retained (∼0.95) and *behavioral fidelity* retained or improved, while every authorship axis collapses to ≈chance—*harness* 0.31→0.08, *model family* 0.20 → 0.12, *identity* 0.83 → 0.09. The properties are separately measurable (double dissociations across six representations), so these are genuinely different axes being moved, not one metric relabeled.

The instance-conditioning is the active ingredient: the construction ranking is strict under both readouts (qubric > verdict questions > generic rubric > free-form summary > raw), reproduces under a local embedder, and no single

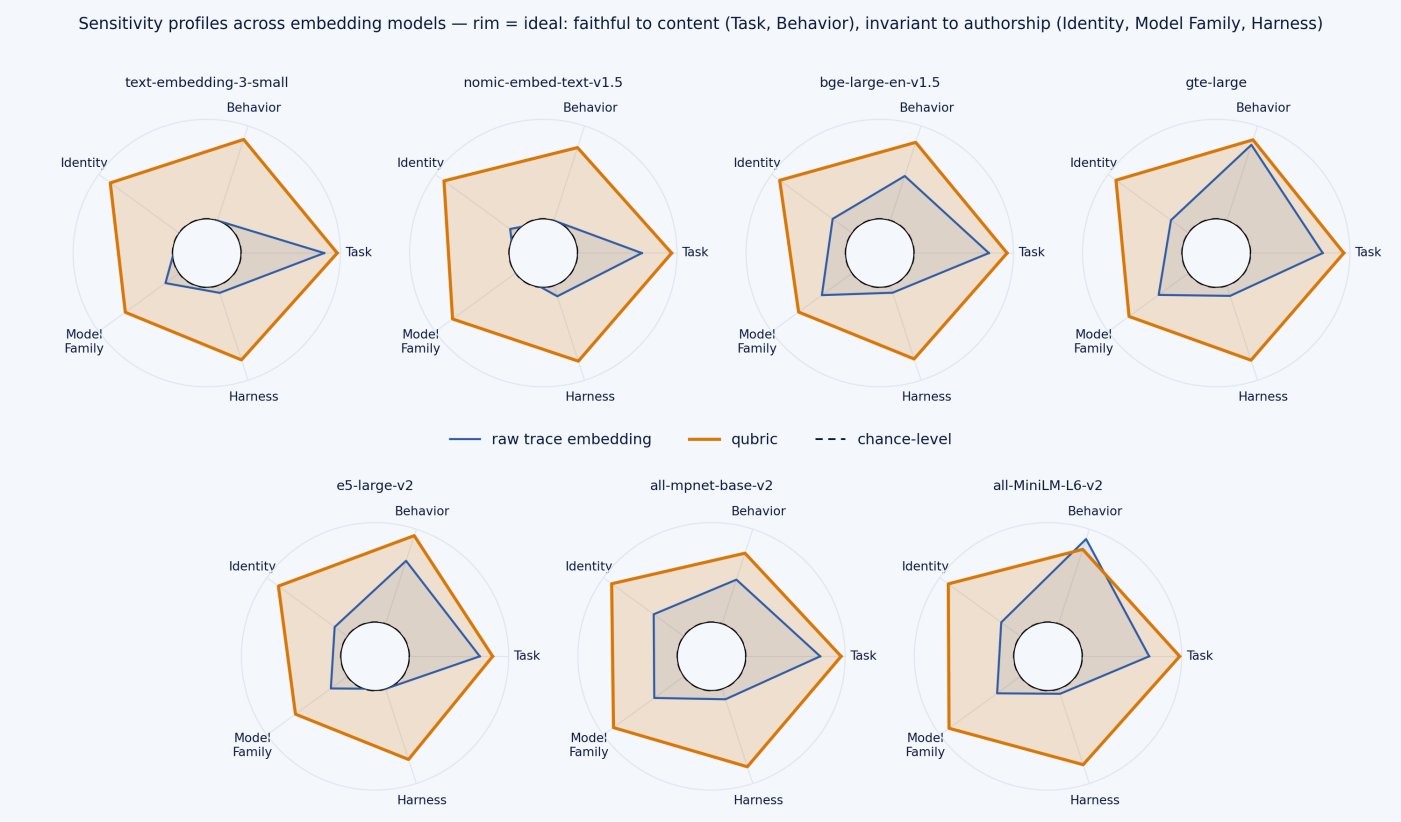

Figure 1: Sensitivity radars across seven embedders. Outside = desirable. Raw embeddings (navy) are authorship-dominated; qubric (amber) collapses Identity / Model family / Harness to chance while holding Task and Behavior.

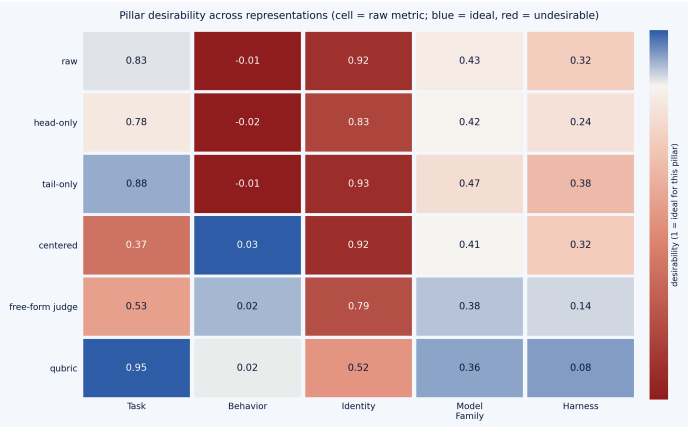

Figure 2: Baselines by pillar: each cell is the raw metric for that axis (columns) under that representation (rows); color is desirability (blue = ideal). Raw-trace variants (head/tail/centered) read authorship; qubric is the only row that is simultaneously task-faithful and authorship-invariant.

rubric section carries the effect (best single section 0.091 vs. 0.078 for the union). Judge choice degrades gracefully: an open frontier judge (DeepSeek-v3.1, 0.083/0.079) matches the closed reference (gpt-5.4-mini, 0.078/0.078) within panel resolution, and a weak judge *with* a qubric outperforms a strong judge without one.

Stability: two judges agree on a single trace only 12–28% of the time, but aggregating ∼20 instances drives system-level agreement to 0.82–0.95; replicate runs of the same agent retrieve each other at 6× chance. Per-trace noisy, per-agent reliable—and per-agent is what evaluation uses.

Query-efficient benchmarking

Qubric representations enter the DKPS/PKPS perspective space; a new agent's full-benchmark resolve rate is predicted from a few probe traces against references' cached coverage, blended with the sample score. Evaluation is leave-one-LLM-out throughout (naive CV inflates accuracy ∼25% via model-family leakage).

probes m	sample score	qubric pipeline	+selection
1	0.44	**0.095**	—
5	0.16	**0.074**	0.057
20	0.06	**0.048–0.053**	0.053

Table 1: MAE predicting true resolve rates (107 systems). One probe trace ≈ 8–10 scored runs.

One trace carries far more than one bit: the strongest correctness-only competitor (CAPA-style error overlap) needs 15–20 *scored* probes to match one *trace* (0.37 MAE at m=1), and trails the ensemble at every budget. The reference pool, not probe count, is the binding resource: error falls 0.145 → 0.078 as the pool grows 5→96, still declining (Fig. 3). Every layer is ablated; CV-inside-selection and PCA-composed-with-selection are documented overfitting traps.

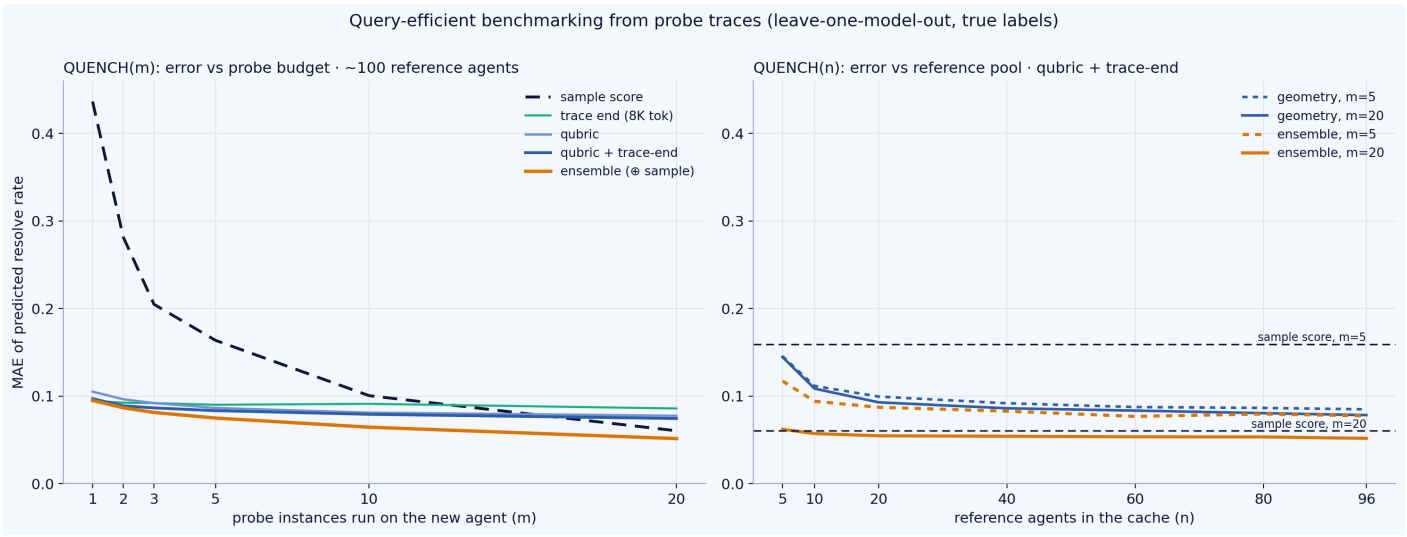

Figure 3: QUENCH: error predicting true resolve rates vs. probe budget m (left) and vs. reference-pool size n (right); leave-one-LLM-out, 107 systems.

Remaining tasks

1. Sensitivity to the choice of Model 1 and Model 2.
2. An additional benchmark, ideally Terminal-Bench (142 systems, 89 tasks, $\geq$5 trials per task, public trajectories).
3. A descriptive figure: trace anatomy and the qubric pipeline.
4. Actual dollar savings from probe-based evaluation vs. full benchmark runs.
5. Literature review.
6. Another downstream task? (Routing? Just needs to beat using the off-the-shelf embedding for this paper.)

Appendix: a worked example (`sympy__sympy-14248`)

A. Example trace (excerpt)

The opening and closing of a sweagent + GPT-4 trace on this instance. Note how much of the text is harness syntax, state dumps, and format echoes rather than content; this particular run ends unsuccessfully at the cost limit.

```
swe_main
First, I'll create a new Python script to reproduce the issue as described. This script will contain the code provided in the issue text. Let'
     s start by creating a file named ‘reproduce_issue.py‘.


action: create reproduce_issue.py

[File: /sympy__sympy/reproduce_issue.py (1 lines total)]
1:

First, I'll create a new Python script to reproduce the issue as described. This script will contain the code provided in the issue text. Let'
     s start by creating a file named ‘reproduce_issue.py‘.

‘‘‘
create reproduce_issue.py
‘‘‘
{"open_file": "n/a", "working_dir": "/sympy__sympy"}

Now I will paste the code from the issue into the ‘reproduce_issue.py‘ file to replicate the bug.


action: edit 1:1
from sympy import *
A = MatrixSymbol('A', 2, 2)
B = MatrixSymbol('B', 2, 2)
print(A - A*B - B)
pprint(A - A*B - B)
print(latex(A - A*B - B))
end_of_edit

[File: /sympy__sympy/reproduce_issue.py (6 lines total)]
1:from sympy import *
2:A = MatrixSymbol('A', 2, 2)
3:B = MatrixSymbol('B', 2, 2)
4:print(A - A*B - B)
5:pprint(A - A*B - B)
6:print(latex(A - A*B - B))
File updated. Please review the changes and make sure they are correct (correct indentation, no duplicate lines, etc). Edit the file again if
      necessary.

Now I will paste the code from the issue into the ‘reproduce_issue.py‘ file to replicate the bug.

‘‘‘
edit 1:1
from sympy import *
A = MatrixSymbol('A', 2, 2)
B = MatrixSymbol('B', 2, 2)
print(A - A*B - B)
pprint(A - A*B - B)
print(latex(A - A*B - B))
end_of_edit
‘‘‘
{"open
[... ~40K tokens omitted ...]
-------------------------
[File: /sympy__sympy/sympy/plotting/plot.py (1828 lines total)]
(22 more lines above)
23:"""
24:
25:from __future__ import print_function, division
26:
27:import inspect
28:from collections import Callable
29:import warnings
30:import sys
31:
32:from sympy import sympify, Expr, Tuple, Dummy, Symbol
33:from sympy.external import import_module
(1795 more lines below)
-------------------------------------------------
Your changes have NOT been applied. Please fix your edit command and try again.
You either need to 1) Specify the correct start/end line arguments or 2) Correct your edit code.
DO NOT re-run the same failed edit command. Running it again will lead to the same error.

(Open file: /sympy__sympy/sympy/plotting/plot.py)
(Current directory: /sympy__sympy)
bash-$
primary
role: assistant
```

```
Exit due to cost limit
Exit due to cost limit
action: exit_cost
primary
```

B. The qubric path: problem → rubric → extraction

The problem statement for this instance (the benchmark task input — it describes the symptom only and does not prescribe a fix):

```
The difference of MatrixSymbols prints as a sum with (-1) coefficient
Internally, differences like a-b are represented as the sum of a with ‘(-1)*b‘, but they are supposed to print like a-b. This does not happen
      with MatrixSymbols. I tried three printers: str, pretty, and latex:
‘‘‘
from sympy import *
A = MatrixSymbol(’A’, 2, 2)
B = MatrixSymbol(’B’, 2, 2)
print(A - A*B - B)
pprint(A - A*B - B)
latex(A - A*B - B)
‘‘‘
Output:
‘‘‘
(-1)*B + (-1)*A*B + A
-B + -A*B + A
’-1 B + -1 A B + A’
‘‘‘

Based on a [Stack Overflow post](https://stackoverflow.com/q/48826611)
```

The rubric-generation prompt (Model 1), with the problem statement above substituted for {`problem`}:

```
Here is a repository issue that a software-engineering agent will be asked to fix:

{problem}

Write an instance-specific rubric for judging an agent’s trace on THIS issue:
for each section below, one sentence saying what specifically to look for in
this instance (name the likely relevant modules/files/tests if inferable).

Sections: understanding, localization, reproduction, editing, verification, final_state.
Respond with ONLY a JSON object with those keys, each a one-sentence string.
```

The resulting query-specific rubric. None of the named locations (`sympy/printing/`, the three printer classes) appear in the problem statement — they are inferred:

```
{
 "understanding": "Look for the agent correctly identifying that MatrixSymbol/MatrixExpr subtraction is being canonically represented as Add
     with negative terms, but the printers should render binary subtraction for matrix expressions just like scalar expressions, likely
     involving SymPy’s printing layer rather than matrix algebra simplification.",
 "localization": "Look for the agent localizing the bug to the SymPy printers for MatrixExpr/MatrixSymbol, especially StrPrinter,
     PrettyPrinter, and LatexPrinter in files under sympy/printing/ (for example printers.py, str.py, pretty/pretty.py, latex.py) and any
     MatrixExpr-specific handling that bypasses standard Add printing.",
 "reproduction": "Look for a minimal reproduction using the exact example A = MatrixSymbol(’A’, 2, 2); B = MatrixSymbol(’B’, 2, 2); print(A -
      A*B - B), pprint(...), latex(...), with observed outputs showing ’-1’ coefficients instead of subtraction, and ideally an added
     regression test capturing these outputs.",
 "editing": "Look for a focused code change that teaches the relevant printers to display Add terms involving MatrixExpr/MatrixSymbol
     negatives as subtraction rather than ’-1*term’ formatting, without changing symbolic semantics or scalar printing behavior.",
 "verification": "Look for the agent running or reasoning about targeted printer tests in SymPy’s test suite\u2014likely tests/printing/
     test_str.py, test_pretty.py, and test_latex.py or matrix-related printer tests\u2014and confirming the example now renders as ’A - A*B -
      B’ / equivalent pretty/latex forms.",
 "final_state": "Look for the final state to include a passing regression test for matrix subtraction printing, no changes to algebraic
     canonicalization of MatrixExpr, and correct output for str, pretty, and latex on the provided example."
}
```

The judge (Model 2) reads the full trace through this rubric and emits one short factual description per section; these six texts are what gets embedded. Note the honest record of an unfinished trajectory:

```
{
 "understanding": "The agent recognized the bug by reproducing the issue with MatrixSymbol subtraction, noting that outputs showed ’-1’
     coefficients instead of subtraction. It understood the problem was in the printing layer and proceeded to fix it without analyzing
     canonical representation details.",
 "localization": "The agent localized the bug to SymPy’s printing module by searching for MatrixSymbol in files and examining latex.py,
     focusing on _print_MatrixSymbol and _print_Add methods in printers like StrPrinter, PrettyPrinter, and LatexPrinter.",
 "reproduction": "The agent created a minimal reproduction script with A = MatrixSymbol(’A’, 2, 2); B = MatrixSymbol(’B’, 2, 2); and printed
     str, pretty, and latex outputs, confirming the issue with ’-1’ coefficients in all printers.",
 "editing": "The agent started editing latex.py by adding a _print_MatrixSymbol method and planned to modify _print_Add, but was diverted to
     fix deprecation warnings in other files like plot.py, leaving the main edit unfinished and causing indentation errors.",
 "verification": "The agent never ran or reasoned about targeted printer tests in the test suite; verification was limited to running the
     reproduction script, and no tests from tests/printing/ were executed or considered.",
 "final_state": "The final state did not include a passing regression test or completed fix; the agent was still addressing deprecation
     warnings and had not finalized changes to ensure correct str, pretty, and latex outputs for matrix subtraction."
```

```
}
```